

Report on AI Memory Chatbot Using LangChain and Groq

Introduction

The AI Memory Chatbot is a simple conversational application developed using Python, LangChain, and the Groq Large Language Model (LLM). The chatbot can answer user questions while maintaining a basic memory of previous conversations using a Python list. This project demonstrates the concepts of saving and retrieving conversation history without using databases or external memory libraries.

Objective

The main objective of this project is to build a chatbot that can:

Answer user questions.

Save the conversation in memory.

Retrieve previous conversations when required.

Demonstrate basic memory implementation using Python.

Technologies Used

Python

LangChain

Groq API

python-dotenv

Visual Studio Code

Working of the Project

The program loads the Groq API key from the .env file.

The Groq language model is initialized using LangChain.

A Python list is created to store the conversation history.

The user enters a question.

The chatbot retrieves previous conversations from memory.

The current question and previous memory are sent to the Groq model.

The chatbot generates and displays an answer.

Both the user's question and the AI's response are saved in memory for future use.

Memory Implementation

The project uses a simple Python list as memory.

Save Memory: Stores every user question and AI response.

Retrieve Memory: Reads the stored conversation history before generating a new response.

Show Memory: Displays all saved conversations.

Clear Memory: Removes all stored conversations from memory.

Features

Interactive chatbot.

Basic memory using a Python list.

Saves every conversation automatically.

Retrieves previous conversations.

Displays stored memory.

Clears memory.

Simple and beginner-friendly implementation.

Code

```
Does not require a database.  
from dotenv import load_dotenv  
from langchain_groq import ChatGroq  
import os  
  
# =====  
# LOAD ENVIRONMENT VARIABLES  
# =====
```

```
load_dotenv()
```

```
api_key = os.getenv("GROQ_API_KEY")
```

```
if api_key is None:  
    print("GROQ_API_KEY not found!")  
    exit()
```

```
# =====  
# LOAD MODEL  
# =====
```

```
llm = ChatGroq(  
    model="llama-3.1-8b-instant",  
    temperature=0.7,  
    api_key=api_key  
)
```

```
# =====  
# BASIC MEMORY  
# =====
```

```
memory = []
```

```
# =====  
# SAVE MEMORY  
# =====
```

```
def save_memory(role, message):
```

```
    memory.append({
        "role": role,
        "message": message
    })
```

```
# =====
# RETRIEVE MEMORY
# =====
```

```
def retrieve_memory():
```

```
    if len(memory) == 0:
        return "No previous conversation."
```

```
    context = ""
```

```
    for item in memory:
```

```
        context += f"{item['role']} : {item['message']}\n"
```

```
    return context
```

```
# =====
# SHOW MEMORY
# =====
```

```
def show_memory():
```

```
    print("\n" + "=" * 50)
    print("        SAVED MEMORY")
    print("=" * 50)
```

```
    if len(memory) == 0:
```

```
        print("Memory is Empty")
```

```
    else:
```

```
        for i, item in enumerate(memory, start=1):
```

```
            print(f"{i}. {item['role']} : {item['message']}")
```

```
        print("=" * 50 + "\n")
```

```
# =====
# CLEAR MEMORY
# =====
```

```
def clear_memory():
```

```
    memory.clear()
```

```
    print("\nMemory Cleared Successfully.\n")
```

```
# =====
# CHATBOT
# =====
```

```
print("=" * 50)
print("        AI MEMORY CHATBOT")
print("=" * 50)
```

```
print("\nAvailable Commands")
print("-----")
```

```
print("memory      -> Show Memory")
print("show memory  -> Show Memory")
print("clear       -> Clear Memory")
print("exit        -> Exit")
print("-----\n")
```

```
while True:
```

```
    question = input("You : ")
```

```
    command = question.strip().lower()
```

```
    # Exit
```

```
    if command == "exit":
```

```
        print("\nGood Bye 😊")
```

```
        break
```

```
    # Show Memory
```

```
    elif command == "memory" or command == "show memory":
```

```
        show_memory()
```

```
        continue
```

```
    # Clear Memory
```

```
    elif command == "clear":
```

```
        clear_memory()
```

```
        continue
```

```
    # Retrieve Memory
```

```
    previous_memory = retrieve_memory()
```

```
    prompt = f"""
```

```
You are a helpful AI assistant.
```

```
Below is the previous conversation.
```

```
{previous_memory}
```

```
Current User Question:
```

```
{question}
```

```
Instructions:
```

1. Use previous conversation if useful.
2. If previous conversation is not useful, answer normally.
3. Keep the answer short.

```
"""
```

```
try:
```

```
    response = llm.invoke(prompt)
```

```
    print("\nAI :", response.content)
    print()
```

```
    save_memory("User", question)
```

```
save_memory("AI", response.content)
```

```
except Exception as e:
```

```
print("Error :", e)
```

Output

AI MEMORY CHATBOT

Available Commands

memory -> Show Memory

show memory -> Show Memory

clear -> Clear Memory

exit -> Exit

You : my name is nandita

AI : Nice to meet you, Nandita. How can I assist you today?

You : what is my name

AI : Your name is Nandita.

You : Show Memory

SAVED MEMORY

1. User : my name is nandita

2. AI : Nice to meet you, Nandita. How can I assist you today?

3. User : what is my name

4. AI : Your name is Nandita.

You : exit

Good Bye 😊

Advantages

Easy to understand and implement.

Suitable for learning LangChain concepts.

Demonstrates basic memory management.

Lightweight and fast.

Conclusion

The AI Memory Chatbot successfully demonstrates the concepts of saving and retrieving memory using a basic Python list along with LangChain and the Groq language model. It is a simple educational project that helps beginners understand how conversational memory works before moving to advanced memory techniques such as databases or vector stores.